

Let us create a Django project. Here are the requirements:
When a customer places an order on an e-commerce app, the warehouse team needs to know which shipping box should be used. Each product has dimensions and weight. Each box has internal dimensions, maximum weight capacity, and cost.
Your task is to design and build a small Django-based system that recommends the most suitable box for an order.
This has to be submitted as a GitHub link and a zip file.
There should also be test cases.

Test run output:

- GitHub Actions link

First we will decide what pages we will have with what functionality, and the tech stack to use alongside.

You can suggest something better. Here is my idea.

1. This assignment is in context of an e-commerce company, but doesn't explicitly say an e-commerce app must also be created. This seems to be an internal app used by the warehouse team. So this functionality can be a feature used in the app.
2. For database, Postgres can be used
3. Deployment is not asked for, but it should be done for quality. I have a free AWS account we can use.
4. We should have login functionality. And user types if relevant or needed
5. Frontend stack is not specified. We can use Django templating language for frontend.
6. This is a time-bound assignment, with 24 hours limit.

My understanding is this.

This Django app will take as input an order placed by a customer. It will analyse the items in the order and suggest a box. We should also have a list of box dimensions we have available. An order should be sent in a single box.

Each order should also have statuses like 'order dispatched', 'to be packed', 'packed', etc. so we can show on the index page.

We will make the assignment step by step.

[Show less](#)

This is a well-scoped assignment. Let me refine your understanding and propose a clean plan before we write a single line of code.

Clarifying the Core Problem

The algorithm is the heart of this assignment. Given an order with N items (each with dimensions + weight), find the **cheapest available box** where:

- All items fit (volume-based, with some 3D bin-packing approximation)
- Total weight $\leq$ box's max weight capacity
- If no single box fits, flag it for manual review

This is fundamentally a **warehouse internal tool**, not a customer-facing storefront. Good call on that.

Proposed Pages & Functionality

Page	URL	Who	What
Login / Logout	<code>/login</code> <code>/logout</code>	All	Auth
Dashboard	<code>/</code>	Warehouse staff	Orders by status (Kanban-style or table)
Order Detail	<code>/orders/<id>/</code>	Staff	Items, recommended box, status actions
Orders List	<code>/orders/</code>	Staff	Filterable list with status badges
Products	<code>/products/</code>	Admin/Manager	CRUD for products + dimensions
Boxes	<code>/boxes/</code>	Admin/Manager	CRUD for box inventory
Simulate / Manual Recommend	<code>/orders/<id>/recommend/</code>	Staff	Re-run box recommendation

User Roles

Two roles keeps it clean and realistic:

- **Warehouse Staff** — can view orders, update status, see box recommendation
- **Manager/Admin** — everything above + manage products, boxes, users

Order Statuses (as a flow)

Pending → To Be Packed → Packed → Dispatched → Delivered
↘ Cancelled

Dashboard groups orders by status column.

Tech Stack

Layer	Choice	Reason
Framework	Django 5.x	Required
Database	PostgreSQL	Your preference, good call
Frontend	Django Templates + Tailwind CDN	Fast, no build step needed
Auth	Django built-in auth + groups	Sufficient for two roles
Box algorithm	Pure Python service class	Testable in isolation
Tests	<code>pytest-django</code>	Better than unittest for Django
CI	GitHub Actions	Runs tests on push
Deployment	AWS EC2 + Gunicorn + Nginx	Free tier, straightforward
Containerization	Docker + docker-compose	Makes EC2 deploy clean, also good signal for reviewers

The Box Recommendation Algorithm

Since items go into **one box**, the approach is:

1. Compute **total weight** of all items → must be $\leq$ box max weight
2. Compute a **bounding box** of all items (sum of heights, max of widths/depths — a reasonable approximation for warehouse use)
3. Filter boxes that satisfy both constraints
4. Return the **cheapest** qualifying box
5. If none fits → flag order as **NEEDS_MANUAL_REVIEW**

This is honest, explainable, and testable — which matters for an assignment.

Project Structure

```
box_recommender/  
├── apps/  
│   ├── accounts/      # login, user roles  
│   ├── orders/        # Order, OrderItem models, status flow  
│   ├── products/      # Product model (dimensions, weight)  
│   ├── boxes/         # Box model  
│   └── recommender/   # Pure algorithm + service layer  
├── templates/  
├── static/  
├── docker-compose.yml  
├── Dockerfile  
├── .github/workflows/ci.yml  
└── manage.py
```

Step-by-Step Build Plan

1. **Project scaffold** — Django setup, Postgres config, Docker
2. **Models** — Product, Box, Order, OrderItem
3. **Recommender service** — algorithm + unit tests
4. **Views & Templates** — Dashboard, Orders, Products, Boxes
5. **Auth & Roles** — login, staff vs manager guards
6. **Status workflow** — transitions, dashboard grouping
7. **Tests** — models, algorithm, views
8. **CI** — GitHub Actions workflow
9. **Deploy** — EC2 + Docker

A few questions before we start coding:

Q: For the box-fitting algorithm, should we use simple bounding-box volume (fast, explainable) or attempt a more realistic 3D bin-packing approach (complex, slower)?

A: Simple bounding box — good enough for a warehouse tool

Q: For AWS deployment, what's your preference?

A: Skip deployment for now, focus on code quality

Q: Should orders be created manually in this app (warehouse staff enters them), or seeded via fixtures/admin for demo purposes?

A: Both

Perfect. Let's build this cleanly and systematically. Here's the final agreed spec before we start:

Final Spec Summary

- **Algorithm:** Bounding box (sum heights, max width/depth across items × quantities)
 - **Deployment:** Skip AWS, focus on code quality + Docker for local dev
 - **Orders:** Manual entry form in app + fixtures/admin for demo seeding
 - **CI:** GitHub Actions running pytest
-

Step 1 — Project Scaffold

We'll create the full project structure, Docker setup, base settings, and empty apps.

Check available skills

Now let's build the full project:

Created 5 files, ran a command

Now the Django config package:

Created 4 files, ran a command

Now the **accounts app** — custom user model with roles:

Created 5 files

Now the **products app**:

Created 5 files

Now the **boxes app**:

Created 5 files

Now the **recommender service** — the core algorithm:

Recommender/service.py — core box recommendation algorithm

Now the **orders app** — the most complex:

Created 7 files

Now the **accounts mixin**:

Now the **fixtures** for demo data:

Created 2 files, ran a command

Now the **tests**:

Created 6 files, ran a command

Now the **GitHub Actions CI**:

.github/workflows/ci.yml — GitHub Actions CI

Now the **templates** — starting with the base layout:

Created 3 files

I am able to run this on my local with sqlite3 with a directory structure different from yours, and similar to the one in Django documentation tutorials.
Secondly, I am using Django 6 and python 3.14 on my local but it shouldn't matter.
Let's move to the next step.
we still aren't using Postgres. (I don't have it on my local either)
I ran the app but on my local, not docker container.
We have to first run the tests before going for deployment, preferably in GitHub actions.

Good, that's a sensible adaptation. Since you're running locally on SQLite (not Postgres, not Docker), let's just make sure everything is consistent and get the test suite green before touching deployment.

A few things to confirm/fix given your setup:

1. Your local settings should use SQLite, not the Postgres config I wrote for `settings.py`. Did you modify `settings.py` to point at SQLite directly, or are you running with `settings_test`? This matters because CI should mirror whatever actually works for you.
2. Django 6 / Python 3.14 — should be fine, but I'd rather verify by actually running the suite than assume.

Let's just run it and see what breaks.

Install dependencies

No network access in this sandbox. so I can't pip install or run pytest here. Since

you're actually running this locally with Django 6 / Python 3.14, let's adjust to match your real environment instead of guessing.

Check what's available locally

No Django installed here either, and no network to fetch it — so I genuinely can't execute the test suite in this sandbox. You're the one with a working local environment; let's use that.

A couple of clarifying things first, since you mentioned a different directory structure (Django-tutorial style):

Q: What does your current project layout look like — closer to `manage.py` at root with a single config-style package (like mine), or did you rename/flatten things (e.g. no `'apps/'` folder, apps directly at root)?

A: Apps moved to project root (no `apps/` wrapper)

Q: For settings, are you using one `settings.py` with SQLite hardcoded, or did you keep `settings.py` (Postgres) + `settings_test.py` (SQLite) split like I wrote?

A: Kept the `settings.py/settings_test.py` split

Q: Have you actually run `pytest` locally yet, or do you want me to first patch the code for Django 6 compatibility before you try?

A: Ran it and it passed, just want CI now

Great — tests pass locally with apps at root level (no `apps/` wrapper) and the `settings.py/settings_test.py` split intact. Let's get GitHub Actions set up to match that.

Since your layout dropped the `apps/` prefix, the CI workflow itself doesn't actually need to know about your internal import paths — it just needs to install deps and run `pytest`. The workflow I wrote earlier should work as-is, but let's tighten it for Django 6/Python 3.14 and add a Postgres service container so CI is closer to a production-like check (even though you dev locally on SQLite).

Q: Should GitHub Actions test against SQLite (matches your local dev, simplest) or spin up a Postgres service container (matches eventual production, more realistic)?

A: Postgres service container — closer to prod

Q: What Python version should CI use to match your local setup?

A: 3.14 (match your local exactly)

Good choices — catches real DB-specific bugs before they bite, and matches your actual runtime exactly.

Since your repo structure changed (apps at root, no `apps/` wrapper) and `settings.py` now points at Postgres while `settings_test.py` handles SQLite, running CI against Postgres means **`settings_test.py` needs a Postgres-pointing variant for CI**, or CI should just use `settings.py` directly with env vars

pointing at the service container. The cleaner approach: keep `settings.py` reading DB config from environment variables (which it likely already does if you followed the pattern), and have CI set those env vars to point at the Postgres service.

A couple of things I need from you since I can't see your actual repo:

Q: Does your `settings.py` read database credentials from environment variables (so CI can override them), or are they hardcoded?

A: Yes, `settings.py` reads DB config from env vars (`os.environ / decouple`)

Q: How do you want to proceed for me to write an accurate CI workflow file?

A: Just generate the workflow, I'll adapt paths myself

Created a file, ran a command

Here's the workflow. A few notes since I don't have your actual repo to verify against — check these before pushing:

`DJANGO_SETTINGS_MODULE: settings_test` — I dropped the `config.` prefix since your apps are at root with no wrapper package. If your `settings_test.py` lives at the project root (sibling to `manage.py`), this is correct. If it's still nested somewhere, adjust the dotted path accordingly.

`settings_test.py` must point at Postgres now, not SQLite — since your test settings currently hardcode SQLite (per what you told me earlier), you'll need to either:

- Make `settings_test.py` read DB config from env vars too (cleanest — just inherit from `settings.py` and only override things like `PASSWORD_HASHERS`), or
- Keep a separate CI-specific settings file

Given you said `settings.py` already reads DB config from env vars, the simplest fix is having `settings_test.py` do `from .settings import *` (or `from settings import *` at root) and only override non-DB things (faster password hasher, `DEBUG=False`, etc.) — letting the CI env vars drive the actual Postgres

connection. Locally without those env vars set, you'd fall back to whatever defaults `settings.py` has.

`pytest.ini` reference — make sure it doesn't pin `DJANGO_SETTINGS_MODULE = config.settings_test` (the old path) since that'll silently override the workflow's env var. Check that file too.

Want me to also patch a `settings_test.py` example reflecting "inherit + override" so you can diff it against your actual file?

I dont have Postgres. I didnt use it before

Got it — let's not introduce a dependency you don't have. Drop the Postgres service container from CI and just run tests against SQLite, matching your actual local setup exactly.

Simplify CI to SQLite-only, matching local dev

☒ Simplify CI to SQLite-only, matching local dev

Script

☑ Done

That's it — no Postgres service, no DB env vars, nothing to install beyond `requirements.txt`. This runs `pytest` using `settings_test.py` (whatever that points at locally — presumably SQLite, since that's what you ran successfully).

Two things worth double-checking against your actual repo before you push:

1. `DJANGO_SETTINGS_MODULE: settings_test` — correct only if `settings_test.py` sits at the project root next to `manage.py` with no package prefix. If it's nested in a folder, this needs that dotted path.
2. `requirements.txt` **shouldn't list** `psycopg2-binary` anymore if you're not using Postgres at all — otherwise CI will try to install it for no reason (it'll still work, just unnecessary). Worth trimming if you genuinely don't need it.

Want me to also strip Postgres references out of the earlier

`settings.py` / `docker-compose.yml` I gave you, so everything in this thread is consistent with SQLite-only, or are you maintaining those files yourself at this point and just need the CI piece?

Just need the CI piece.

That's done — the CI workflow above is SQLite-only, Python 3.14, just installs `requirements.txt` and runs `pytest`. Push it under `.github/workflows/ci.yml` and it should mirror your local run exactly.

```
running into this problem in the git ci even tho migrations ran
successfully.
django.db.utils.OperationalError: Problem installing fixture
'/home/runner/work/warehouse/warehouse/boxes/fixtures/boxes/bo
xes.json': Could not load boxes.Box(pk=1): no such table: boxes_box
Error: Process completed with exit code 1.
```

here's my file

Show more

A few things going on here, and worth untangling them in order:

This file isn't the SQLite-only one we agreed on — it's reintroduced a Postgres service container, even though you told me you don't have Postgres and don't use it. That's why this is confusing: the `migrate` and `loaddata` steps are running against *different databases*.

The actual bug: `migrate` runs fine, but `loaddata` fails with "no such table" — that's the signature of Django connecting to **two different databases in two different steps**. Most likely: your `settings_test.py` doesn't read `DB_ENGINE` / `DB_HOST` etc. from env vars the way you think it does, so `migrate` created tables in SQLite (default fallback) while `loaddata` — or vice versa — ran against the empty Postgres container. Each `run:` step is also a **fresh shell**, but that's not the issue here since env vars at job level persist across steps; the issue is almost certainly that `settings_test.py` hardcodes SQLite (as you set it up originally) and ignores the `DB_*` env vars entirely, so `migrate` works fine against a fresh SQLite file — but then something about fixture loading or a second settings resolution points elsewhere.

Rather than debug Postgres-vs-SQLite plumbing for a database you don't even use, let's go back to the simpler, correct version: no Postgres service, no

`DB_ENGINE` overrides. Just SQLite via `settings test`. and drop `loaddata` from

CI entirely — fixtures are for local demo data, not test runs (your `pytest` suite uses factories, not these fixtures).

Revert to SQLite-only CI, drop Postgres service and loaddata steps

Revert to SQLite-only CI, drop Postgres service and loaddata steps

Script

Done

Changes from your version:

1. **Removed the `postgres:` services block entirely** and the `DB_ENGINE` / `DB_HOST` /etc. env vars — you don't have Postgres, so don't make CI pretend to.
2. **Removed `psycpg2-binary` install** — not needed without Postgres.
3. **Removed `makemigrations`** — CI should never generate migrations; it should only apply migrations already committed to your repo. If `makemigrations` finds nothing to write it's harmless, but it's the wrong instinct for CI (it can mask a forgotten/uncommitted migration locally).
4. **Removed `loaddata` steps** — this was the actual cause of your error, and it's also unnecessary. Your `pytest` suite uses `factory_boy` factories (per `tests/factories.py`), not these JSON fixtures. The fixtures are for `python manage.py loaddata` when you seed a local demo database, not for the test run. Mixing the two is what created the table-mismatch.
5. **Set `DJANGO_SETTINGS_MODULE: warehouse.settings_test`** matching your project name (`warehouse`) — adjust if your settings module path differs.

One thing to verify: does `warehouse/settings_test.py` exist and import cleanly from `warehouse/settings.py` with SQLite as the `DATABASES` default? If `settings_test.py` falls back to reading `DB_ENGINE` from env and there's none set now, double check it doesn't error out — it should just hit your hardcoded SQLite default.

